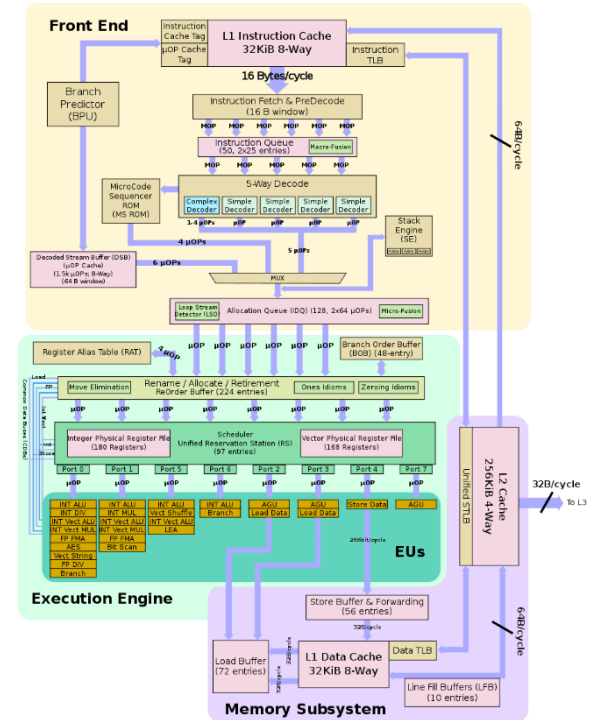


David Oswald

Code:

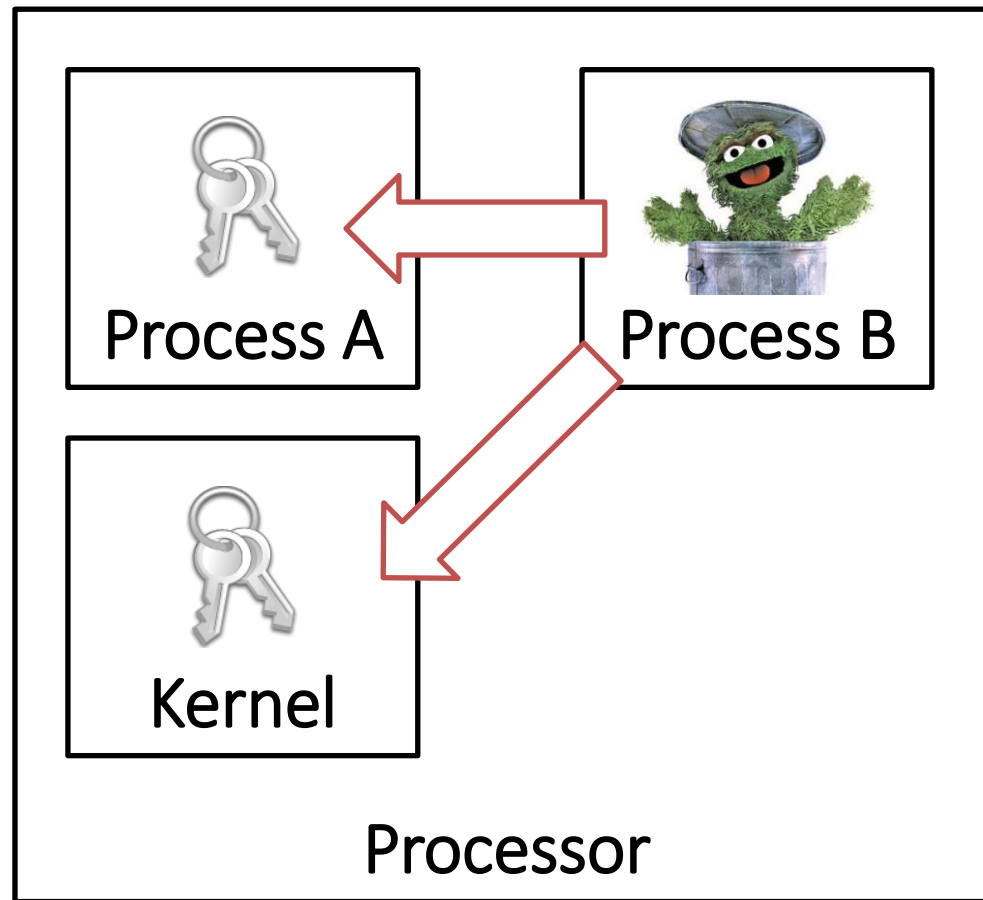


Goals of this lecture

After this lecture, you will ...

- Understand how (some) attacks based on transient execution work
- See how performance optimizations can break security
- Know why it's important to understand the interaction between architecture and microarchitecture

Attack setup



A short history of side-channel attacks

1996: Timing attacks (Kocher)

1998: Differential Power Analysis and Simple Power Analysis
by Kocher et al.

... various improvements and countermeasures ...

2005: “Cache-timing attacks on AES” (Bernstein)

... some more software side-channel attacks ...

2014: “FLUSH+RELOAD: A High Resolution, Low Noise, L3 Cache
Side-Channel Attack” (Yarom et al.)

A short history of side-channel attacks

1996: Timing attacks (Kocher)

1998: Differential Power Analysis and Simple Power Analysis
by Kocher et al.

... various improvements and countermeasures ...

2005: “Cache-timing attacks on AES” (Bernstein)

... some more software side-channel attacks ...

2014: “FLUSH+RELOAD: A High Resolution, Low Noise, L3 Cache Side-Channel Attack” (Yarom et al.)

2018: Meltdown / Spectre

Who reported Meltdown?

Meltdown was independently discovered and reported by three teams:

- [Jann Horn](#) (Google.Project.Zero),
- [Werner Haas, Thomas Prescher](#) (Cyberus.Technology),
- [Daniel Gruss, Moritz Lipp, Stefan Mangard, Michael Schwarz](#) (Graz.University.of.Technology)

Who reported Spectre?

Spectre was independently discovered and reported by two people:

- [Jann Horn](#) (Google.Project.Zero) and
- [Paul Kocher](#) in collaboration with, in alphabetical order, [Daniel Genkin](#) (University.of.Pennsylvania and University.of.Maryland), [Mike Hamburg](#) (Rambus), [Moritz Lipp](#) (Graz.University.of.Technology), and [Yuval Yarom](#) (University.of.Adelaide and Data61)

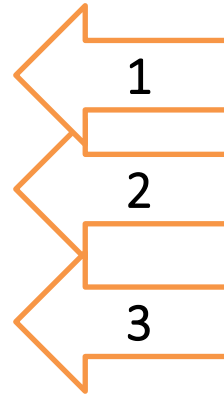
An initial question...

How does this code execute?

```
int p1 = a * b;
```

```
int p2 = c * d;
```

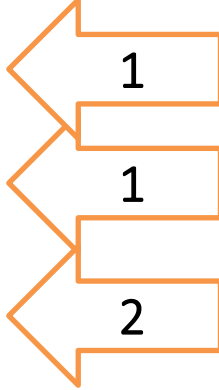
```
return p1 + p2;
```

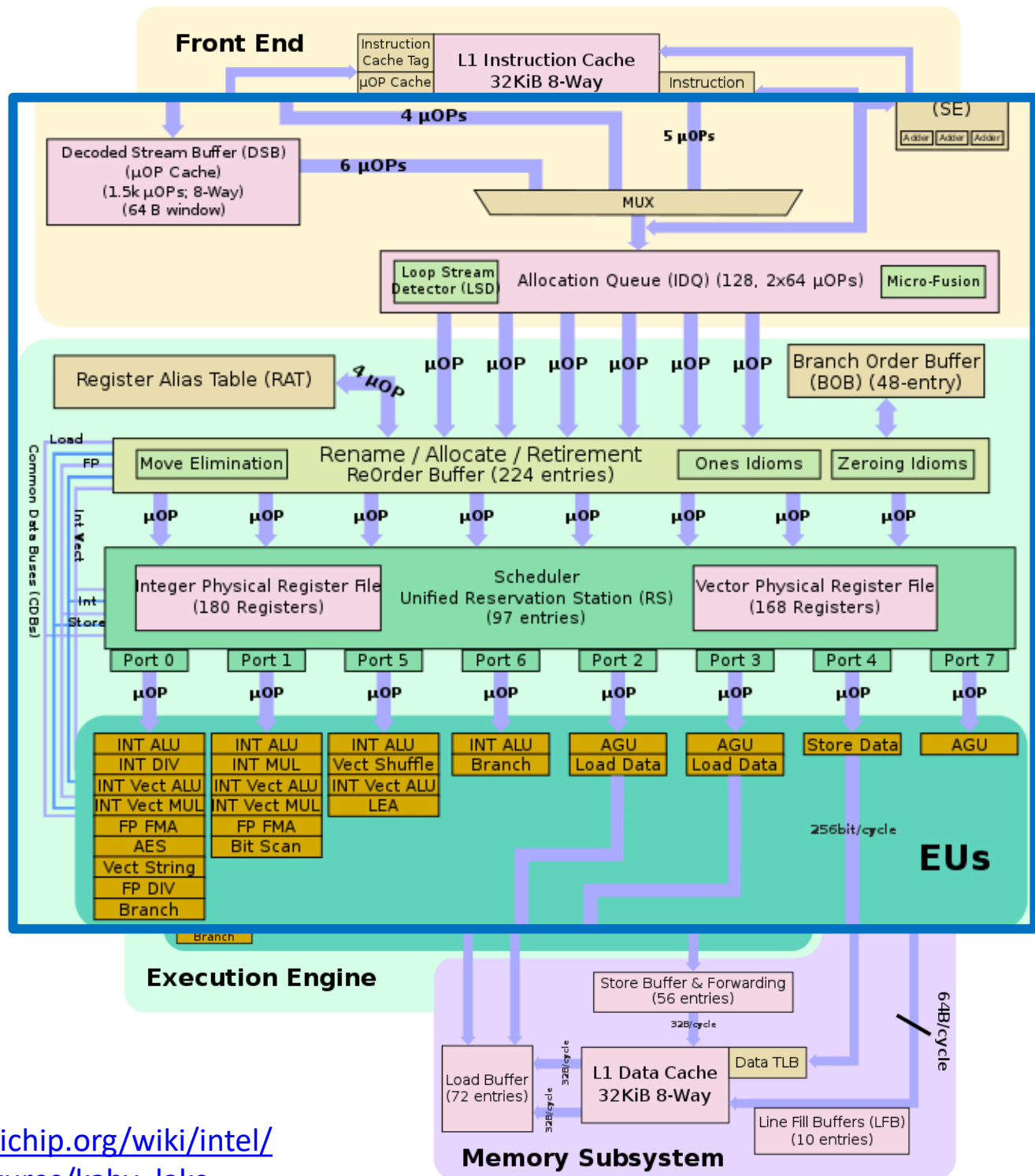


What really happens

How does this code execute?

```
int p1 = a * b;  
int p2 = c * d;  
return p1 + p2;
```

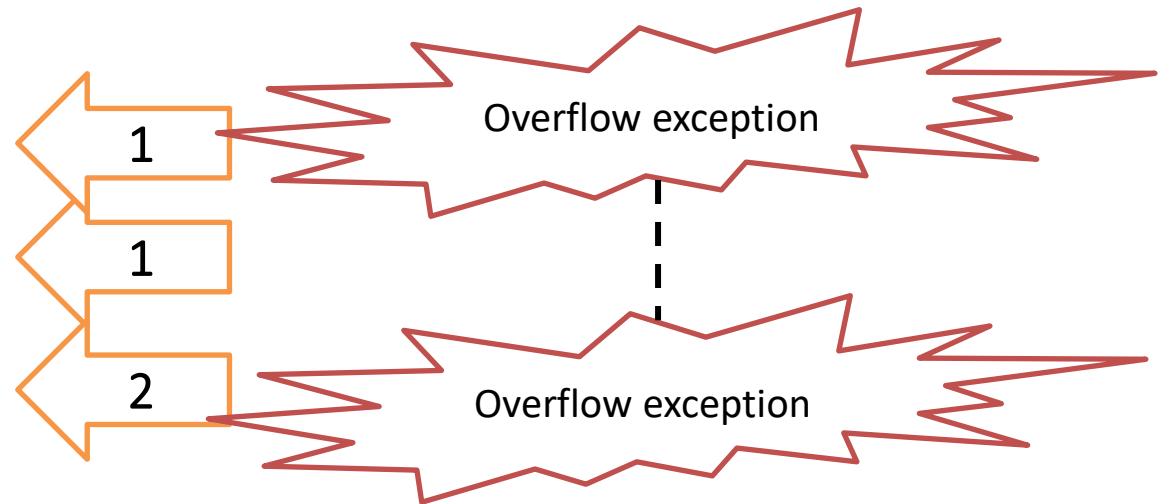




What happens on error?

How does this code execute?

```
int p1 = a * b;  
int p2 = c * d;  
return p1 + p2;
```



Live-Demo

“Anything that can go wrong,
will go wrong”

Out-of-order execution

What did just happen?

```
maccess(p1);
```

```
// This is evil!  
dummy = *NULL;
```

```
maccess(p2);
```

Architecturally this is not happening
– but *micro-architecturally* this
leaves traces (e.g. cache)

The Meltdown attack



<https://meltdownattack.com/meltdown.pdf>

The idea of meltdown

```
uint8_t probe[256 * 4096];
```

```
flush(probe)
```

```
// This will fault  
x = *kernel_addr;
```

```
probe[x * 4096] += 1;
```

```
// suppress fault
```

```
x = reload(probe)
```

Architecturally this is not happening
– but *micro-architecturally* this
leaves traces (cache)

Core steps of Meltdown

1. Load from kernel address succeeds *transiently*, but later raises an exception
2. Use the loaded byte x to index into probe array
3. The exception arrives later, CPU **rolls everything back ...**
4. ... but the cache state **reveals** x (using Flush+Reload)

Live-Demo

“Anything that can go wrong,
will go wrong”

Meltdown

Technical remarks

<https://github.com/IAIK/meltdown>



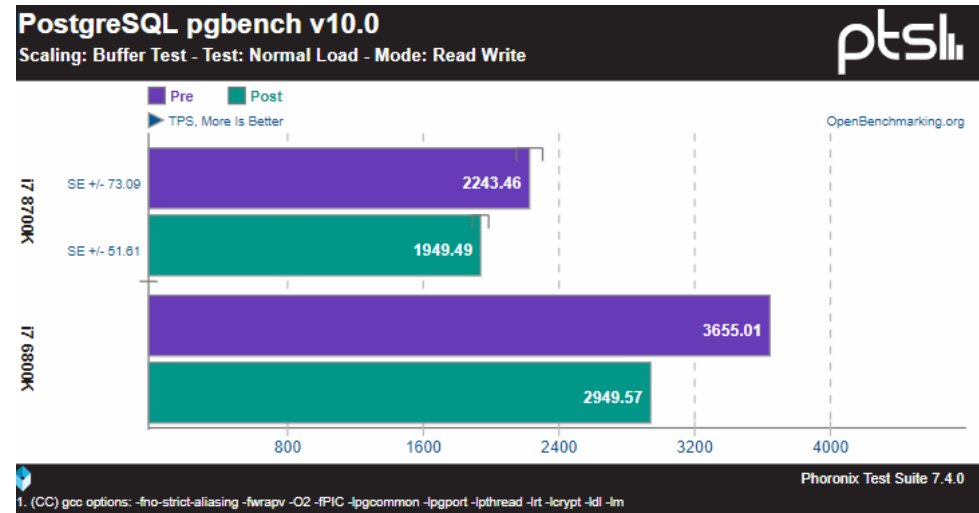
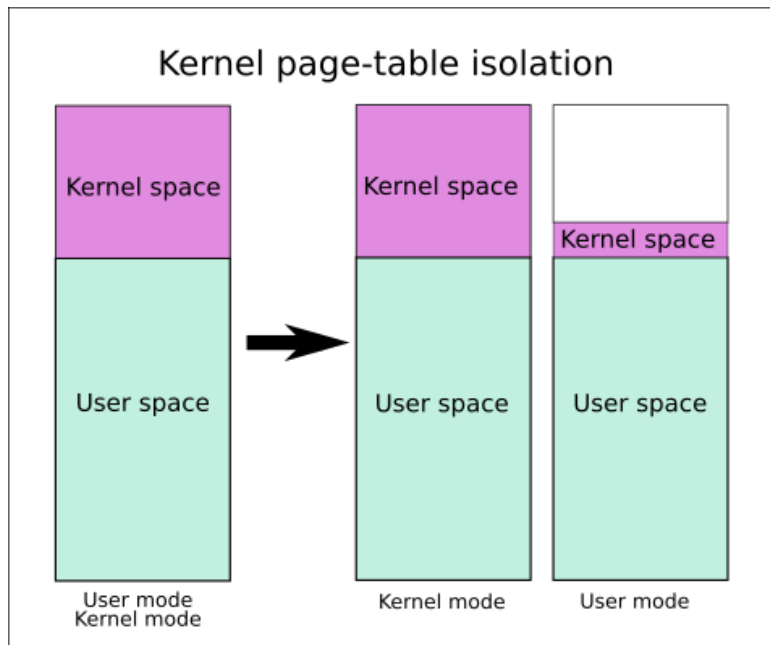
- Exception suppression
 - Custom signal handler + setjmp
 - Intel TSX
- Targeted data must be cached
- Reduce errors by repeating many times and selecting most frequent x
- Various small tweaks

Impact

- Almost every Intel CPU before 2018
- AMD not affected
- ARM: Cortex-A75
- IBM: System Z, Power Architecture, POWER8 and POWER9
- Apple: All Mac and iOS devices

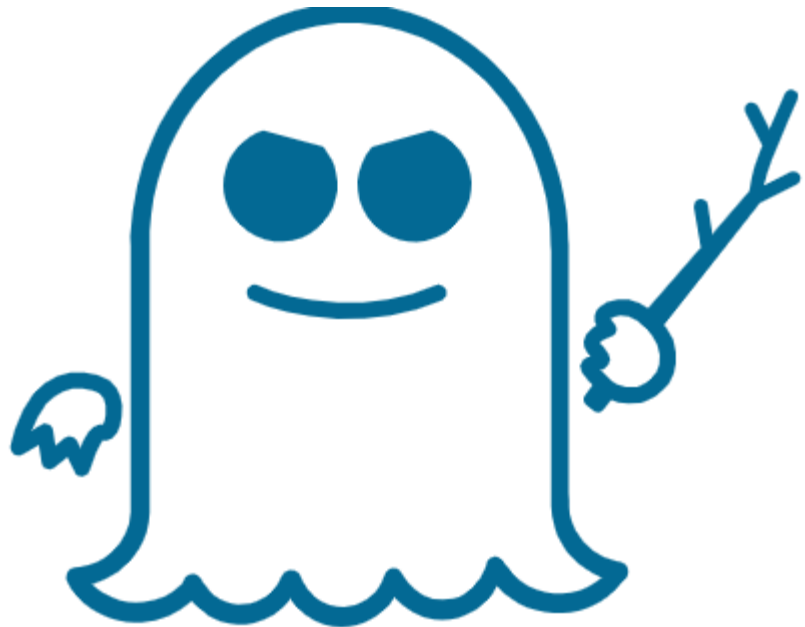
Fixes

- New CPUs have hardware fixes
- Old CPUs need kernel patches:



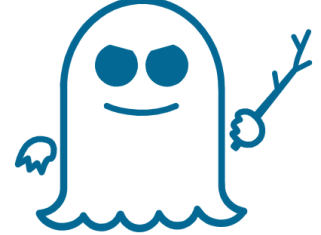
<https://www.phoronix.com/scan.php?page=article&item=linux-415-x86pti&num=2>

The Spectre attack



<https://spectreattack.com/spectre.pdf>

The idea of Spectre



22

```
// Assume array1 has N elements
if(x < N)
{
    y = array2[array1[x] * 4096];
}
```

Get this to transiently
execute with $x \geq N$

The branch predictor

```
if(x < N)
{
    y = array2[array1[x] * 4096];
}
```

- The CPU tries to guess whether the if branch is taken, and transiently executes it (or not)
- We can train the branch predictor:
 - Execute many times with $x < N$
 - Now execute once with $x \geq N$
- This will leak OOB read in the cache

Spectre – technical remarks

- We saw a *transient* buffer overflow
- Spectre uses specific *gadgets* to leak memory
- There are many variants
- Virtually any CPU with speculative execution is affected
- Countermeasures are often specific

Alternatively, the software could be modified to use *serializing* or *speculation blocking* instructions that ensure that instructions following them are not executed speculatively. Intel and AMD recommend the use of the `lfence` instruction [4, 36]. The safest (but slowest) approach to protect conditional branches would be to add such an instruction on the two outcomes of every conditional branch. However, this amounts

Take-home points

- Tom lied to you 😊 – checking bounds against buffer overflows is not really enough
- There is now a variety of transient execution attacks and lots of research
- Speculative execution is fundamental to modern processor architecture
- Meltdown has been fixed
- But attacks like Spectre are hard to prevent generically

Thanks!

Questions / discussion?

d.f.oswald@bham.ac.uk